

Retail Demand Forecasting with an End-to-End Machine Learning Monitoring Pipeline

*Forecasting Retail Sales and Monitoring Model Performance in Production

Buse Özbek

*School of Computer Science and Information Technology
Lucerne University of Applied Sciences and Arts (HSLU)
Lucerne, Switzerland*

buse.ozbek@stud.hslu.ch
github.com/buse-ozbek/artifin

June 2026

Abstract

This project builds a complete, reproducible machine-learning system for forecasting daily retail demand (*Units Sold*) and, more importantly, for *operating* that model in a production-like setting. Three regression models (Linear Regression, Random Forest, and Gradient Boosting) are trained, tracked, and compared with MLflow; the best model is registered and served through a FastAPI endpoint; the flow is scheduled with Prefect; and a monitoring layer measures data drift (Kolmogorov–Smirnov test) and regression performance over simulated production batches, storing everything in PostgreSQL for visualisation in Grafana. The deployed model (a Gradient Boosting regressor, registered as `retail_demand_model` version 18 under the `Staging` alias) attains a very high goodness of fit on held-out data ($R^2 \approx 0.994$, $MAE \approx 7.2$ units). This report makes two linked findings. First, the dataset ships with an operational **Demand Forecast** column, which is itself strongly predictive of realised demand ($r \approx 0.997$), so the modelling objective is framed as building a learned model that *improves on, and stress-tests, that operational forecast* rather than predicting demand from scratch. The learned model does beat the naive “trust the forecast” rule (MAE 7.0 vs 8.3) by combining the forecast with contextual features, and the analysis quantifies exactly how much additional signal those features carry on this synthetic dataset. Second, and this is what the monitoring layer reveals, the model has genuinely learned to rely on the contextual features: when drift is injected into them (Price, Discount, Inventory Level, Competitor Pricing), accuracy degrades sharply and monotonically with drift severity, with MAE rising from ≈ 7 to as high as ≈ 67 and R^2 falling from ≈ 0.99 to ≈ 0.38 . The contribution of this work is therefore a working demonstration of the *MLOps* lifecycle: experiment tracking, a model registry, serving, orchestration, a tested CI/CD pipeline, and a drift-monitoring loop that demonstrably catches degradation on a realistic retail dataset.

1 Introduction

Accurate demand forecasting underpins inventory management, replenishment, and supply-chain planning in retail. Under-forecasting causes stock-outs and lost sales; over-forecasting ties up capital and storage. The modelling task in this project is to predict the number of units of a product sold in a given store on a given day. A distinctive property of the *Retail Store Inventory Forecasting* dataset used here is that it already contains a **Demand Forecast** column that is near-perfectly correlated with the target. The modelling objective is therefore framed accordingly: rather than predicting demand from scratch, the goal is to learn a model that *improves on that*

column as a baseline by exploiting the remaining contextual features (price, discount, inventory, weather, promotions, seasonality), and then to put that model through a full production lifecycle and stress-test it under drift.

A model that performs well in an offline experiment is only the beginning of a useful system. In practice, the data a model sees in production drifts away from the data it was trained on, and silent degradation is one of the most common failure modes of deployed ML systems [7]. The aim of this project is therefore deliberately broader than achieving a good test score: it is to assemble the surrounding machinery that makes a model *operable* (tracked, versioned, served, scheduled, automatically tested, and continuously monitored) and then to demonstrate that the monitoring layer actually does its job when the inputs change.

Concretely, the system is built as three pipelines: a **training** pipeline, a **deployment** pipeline (serving, orchestration, containerisation, and CI/CD), and a **monitoring** pipeline. Sections 2–4 cover background, data, and the design of these pipelines; Section 5 reports and, critically, *interprets* the results, including how much the contextual features add over the operational forecast and how the model behaves under drift; Sections 6–7 discuss limitations and next steps.

2 Background and Related Work

Models. Three increasingly expressive regressors are compared. Linear Regression provides an interpretable baseline. Random Forests reduce variance by averaging many de-correlated decision trees [2]. Gradient Boosting builds an additive ensemble in which each tree corrects the residuals of its predecessors [3]; it is typically the strongest of the three on tabular data, at the cost of more hyper-parameters.

MLOps and lifecycle tooling. Managing experiments, artefacts, and model versions by hand does not scale. MLflow addresses this by recording parameters, metrics, and models, and by exposing a model registry that decouples *training* a model from *promoting* one to a serving alias [8]. The need for such discipline is documented by Sculley et al. [7], who argue that the code that trains a model is a small fraction of a real ML system, and by Breck et al. [1], who propose a rubric for production readiness.

Monitoring and drift. Once deployed, a model must be watched. *Data drift*, defined as a change in the distribution of the inputs, can silently erode accuracy [4]. This project detects drift per feature with the two-sample Kolmogorov–Smirnov (KS) test [6], which compares the empirical cumulative distributions of a reference sample and a current batch and reports a p -value; a small p -value indicates the two samples are unlikely to come from the same distribution. Complementary practice is to track live model performance and prediction statistics over time [5], which is exactly the role of the **metrics** and **prediction_logs** tables described below. A central methodological point of this report is that input-drift detection and output-quality monitoring are *complementary, not interchangeable* signals; the experiments below make the relationship between them concrete.

3 Data

The dataset contains **73,100** daily records across multiple stores and products. After feature engineering the model uses 15 inputs:

- **Identifiers / categoricals:** Store ID, Product ID, Category, Region, Weather Condition, Seasonality.
- **Numerical context:** Inventory Level, Demand Forecast, Price, Discount, Competitor Pricing, Holiday/Promotion (binary).

- **Date-derived:** Year, Month, Day-of-Week (extracted from the `Date` column).

Target. *Units Sold* (integer), with mean ≈ 136.5 and standard deviation ≈ 108.9 units across the dataset (range 0–499).

Pre-processing. Categorical columns are integer-encoded with scikit-learn’s `LabelEncoder`; numerical columns are standardised inside the model pipeline with `StandardScaler`. A subtle but important consistency requirement is that the serving code in `app.py` must reproduce the exact same encoding the model saw at training time. Because `LabelEncoder` assigns codes in alphabetical order, the hard-coded category maps in the API (e.g. `Clothing`→0, `Electronics`→1, ...) match the training encoding; store and product identifiers are recovered by extracting the numeric suffix. Any mismatch here would corrupt predictions silently, a classic train/serve skew risk that Section 6 returns to.

4 System Design and Methodology

The system is organised as three pipelines: a **training** pipeline (Section 4.1), a **deployment** pipeline that serves, schedules, containerises, and continuously tests the model (Section 4.2), and a **monitoring** pipeline that watches the served model for drift and degradation (Section 4.3). Figure 1 details the monitoring pipeline, which is the most elaborate of the three.

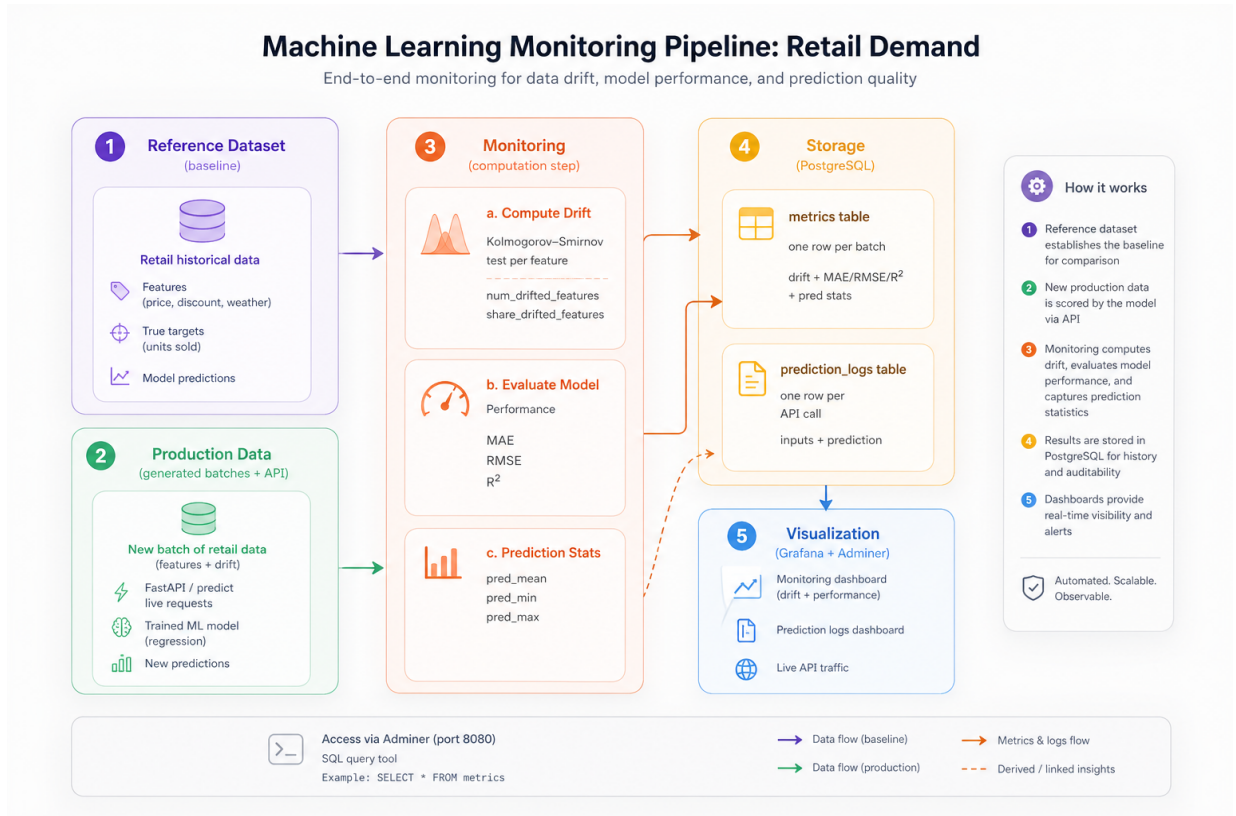


Figure 1: The monitoring pipeline. A reference (baseline) dataset and incoming production batches are scored by the registered model; a monitoring step computes drift, regression performance, and prediction statistics; results are persisted to PostgreSQL and visualised in Grafana, with Adminer for ad-hoc SQL.

4.1 Training pipeline

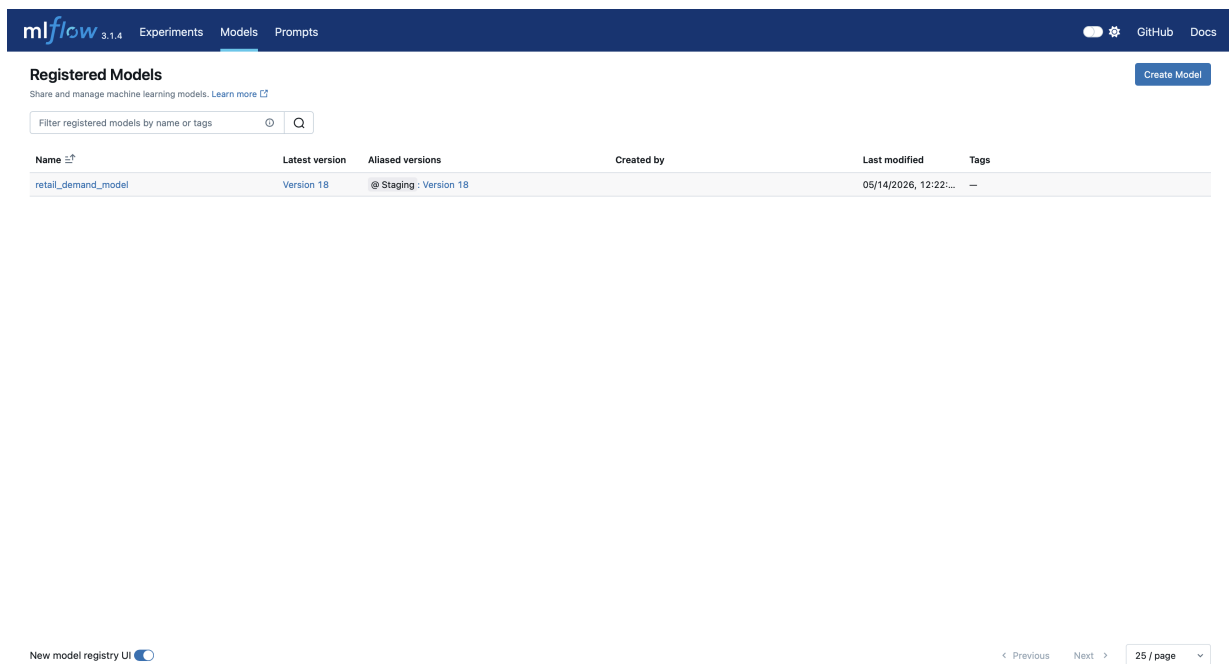
`pipeline.py` loads and pre-processes the data, performs an 80/20 train/test split, and trains the three models, each wrapped in a `Pipeline(StandardScaler → estimator)`. Every run is logged

to MLflow with its parameters, the three metrics (MAE, RMSE, R^2), and the serialised model artefact (Figure 2). The best model is chosen by a simple **voting scheme**: each model earns one point for winning each of the three metrics, and the model with the most points is selected. The winner is then registered in the MLflow Model Registry under the alias **Staging**, so that downstream components reference a model *by role* rather than by file path:

```
result = mlflow.register_model(model_uri, name="retail_demand_model")
client.set_registered_model_alias(
    name="retail_demand_model", alias="Staging", version=result.version)
```

Run Name	Created	Dataset	Duration	Source	Models
Exp3_GradientBoosting	22 days ago	-	9.1s	pipeline...	retail_demand_mo... +1
Exp2_RandomForest	22 days ago	-	2.3s	pipeline...	model
Exp1_LinearRegression	22 days ago	-	1.7s	pipeline...	model
Exp3_GradientBoosting	1 month ago	-	17.7s	deploy.py	model
Exp2_RandomForest	1 month ago	-	2.8s	deploy.py	model
Exp1_LinearRegression	1 month ago	-	2.0s	deploy.py	model
Exp3_GradientBoosting	1 month ago	-	18.0s	deploy.py	model
Exp2_RandomForest	1 month ago	-	2.7s	deploy.py	model
Exp1_LinearRegression	1 month ago	-	1.8s	deploy.py	model
Exp3_GradientBoosting	1 month ago	-	18.2s	deploy.py	model
Exp2_RandomForest	1 month ago	-	2.7s	deploy.py	model
Exp1_LinearRegression	1 month ago	-	1.8s	deploy.py	model
Exp3_GradientBoosting	1 month ago	-	19.7s	deploy.py	model
Exp2_RandomForest	1 month ago	-	19.7s	deploy.py	model
Exp1_LinearRegression	1 month ago	-	19.5s	deploy.py	model
Exp3_GradientBoosting	1 month ago	-	5.4s	deploy.py	model
Exp2_RandomForest	1 month ago	-	5.4s	deploy.py	model
Exp1_LinearRegression	1 month ago	-	5.4s	deploy.py	model
Exp3_GradientBoosting	1 month ago	-	2.2s	deploy.py	model
Exp2_RandomForest	1 month ago	-	2.2s	deploy.py	model

Figure 2: MLflow experiment tracking. All runs of the three models are logged under the `Retail_Demand_Forecasting` experiment with parameters, metrics, and source. The most recent Gradient Boosting run carries the registered-model badge, indicating it is the version promoted to the registry.



Name	Latest version	Aliased versions	Created by	Last modified	Tags
retail_demand_model	Version 18	@Staging: Version 18		05/14/2026, 12:22:...	—

Figure 3: MLflow Model Registry. The Gradient Boosting winner is registered as `retail_demand_model`; **version 18** holds the `@Staging` alias and is the single source of truth consumed by the API, the batch predictor, and the monitoring scripts.

4.2 Deployment pipeline

The deployment pipeline takes the registered model and makes it usable, repeatable, and trustworthy through four components: serving, orchestration, containerisation, and CI/CD.

Serving. `app.py` is a FastAPI service that loads `models:/retail_demand_model@Staging` once at start-up and exposes a typed `POST /predict` endpoint (Figure 4). Each request is validated by a Pydantic schema, pre-processed identically to training, scored, and clipped to be non-negative (demand cannot be negative). For monitoring, *every prediction is logged* to the PostgreSQL `prediction_logs` table. The logging call is wrapped in its own `try/except` (the code explicitly notes that “logging failure should never break the API”), so that if the database is unreachable the prediction is still returned and only a warning is printed. Figure 5 shows a live call returning a prediction of 146 units for a sample Electronics request.

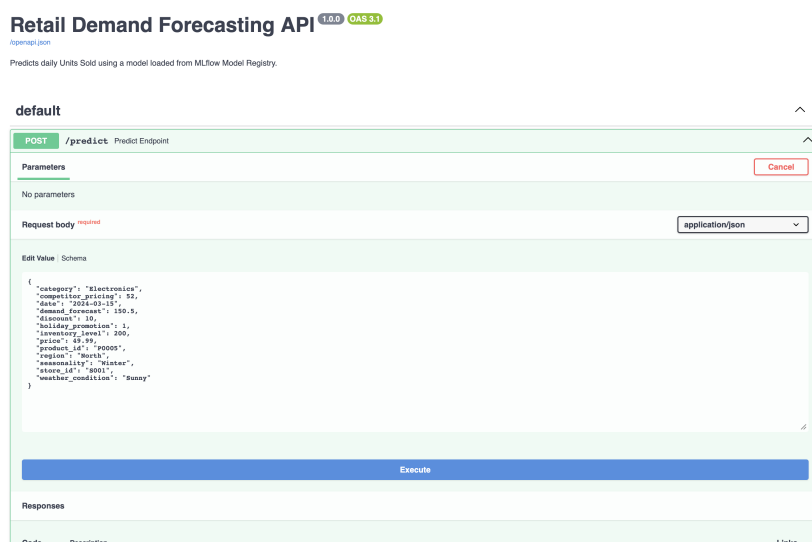


Figure 4: The FastAPI `/predict` endpoint with its auto-generated Swagger UI and the Pydantic-validated request body.

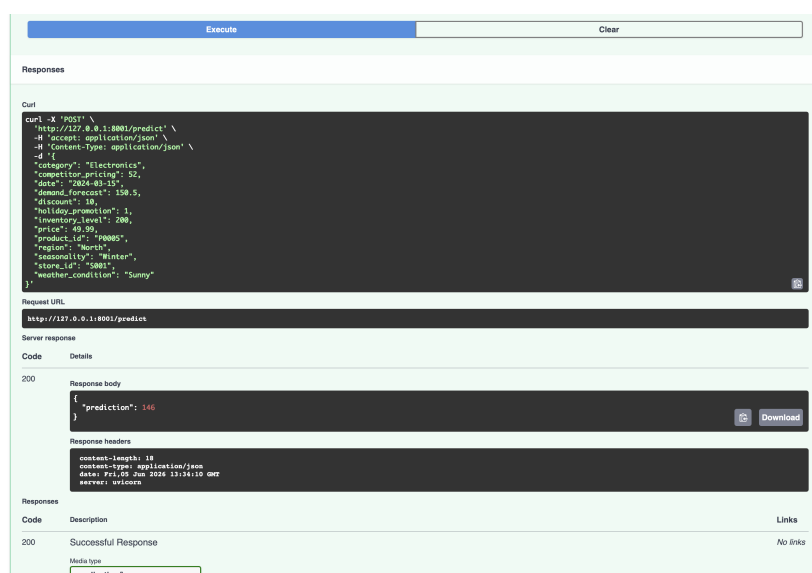


Figure 5: A successful `/predict` call. The service returns `{"prediction": 146}` (HTTP 200) and writes the corresponding row to `prediction_logs`.

Orchestration and scheduling. `train_predict_scheduled.py` re-expresses the training logic as Prefect tasks (load, split, train-and-log, select-best, register) with retries, and `deploy.py` schedules the flow to run daily (cron 35 14 * * *). This turns a manual script into an automated, observable, retrainable job.

Containerisation. The runtime services are packaged with Docker. A `docker-compose.yml` brings up the supporting infrastructure (PostgreSQL, Adminer, and Grafana) while the FastAPI service is built into its own image and run against the same database, so the entire stack can be reproduced with a couple of commands.

Continuous integration and delivery (CI/CD). CI/CD is the central engineering deliverable of the project. To make the service testable, the pure prediction logic is factored into a dependency-light core module (`prediction_core.py`) that is separate from the FastAPI service; the tests exercise that core with a lightweight `DummyModel` stub, so the suite runs in continuous

integration without a live MLflow server or any model artefact. The tests validate *domain-meaningful constraints*: that predictions fall within the project’s valid output range and satisfy logical relationships, which is a robust contract for a regression service. Test correctness is itself verified by deliberately breaking a piece of logic (for example, flipping a categorical encoding) and confirming that the corresponding test fails. A GitHub Actions workflow (`ci-cd.yml`) runs the `pytest` suite on every push, and the environment is pinned to a single Python version across local development, CI, and Docker to avoid the hard-to-debug mismatches that an inconsistent interpreter would introduce.

4.3 Monitoring pipeline

The monitoring pipeline follows a reference-vs-current design, with a deliberately temporal split suggested during the project: the model is trained on the earliest period in the data (year 2022), whose distribution defines the *reference* baseline, and later periods (2023–2024) play the role of incoming production data whose drift we wish to detect.

1. `prepare_reference.py` scores the reference (training-period) data with the registered model and stores features + target + prediction as `reference.csv`.
2. `generate_batch.py` draws incoming batches representing the later periods. A purely temporal split, however, produced almost no measurable degradation here, because this dataset is synthetic and its feature distributions are near-uniform across years. To give the monitor something meaningful to detect, the script therefore adds an explicit *drift-injection* layer on top of the temporal split: it perturbs a random subset of {Price, Discount, Inventory Level, Competitor Pricing}, scaling or shifting each by a random factor, before scoring the perturbed batch, and writes it to `data/current_batches/`. Because the subset and magnitudes vary, successive batches span a controllable spectrum from near-baseline to heavily drifted.
3. `calculate_metrics.py` compares the latest batch to the reference. It runs a Kolmogorov–Smirnov test *per feature* (`ks_2samp(reference[f], current[f])` for each of the 15 features, counting those with $p < 0.05$ as drifted) and, *separately*, computes the regression metrics (MAE, RMSE, R^2) of the batch’s predictions against its targets, plus prediction-distribution statistics. All of these are written as one row to the PostgreSQL `metrics` table. (The KS test is thus a per-feature *input*-drift signal; the regression metrics are a separate *output*-quality signal, a distinction that becomes important in Section 5.)

Grafana reads the two tables to render a monitoring dashboard (drift + performance over time) and a prediction-logs dashboard (live API traffic), with Adminer available for ad-hoc SQL.

5 Results

This section reports the offline model performance, explains *why* that performance is so high, and then presents the monitoring behaviour, which, read together with the offline analysis, tells a more interesting story than either does alone.

5.1 Offline model performance

Table 1 summarises the three experiments on the held-out test set. The voting scheme awarded all three metric wins to **Gradient Boosting**, which was registered as `retail_demand_model` version 18 and promoted to **Staging** (Figure 3). The *Selected (Staging)* row therefore reproduces the Gradient Boosting figures by construction: the served model *is* that run.

Table 1: Test-set performance. The first row is the *operational baseline*: using the dataset’s **Demand Forecast** column directly as the prediction, with no model. The three learned models all improve on it; Gradient Boosting wins all three metrics and is registered under **@Staging**.

Model	MAE	RMSE	R ²
<i>Operational Baseline (no model)</i>	<i>8.3052</i>	<i>10.009</i>	<i>0.9916</i>
Exp1 – Linear Regression	7.4437	8.6298	0.9937
Exp2 – Random Forest	7.2394	8.5035	0.9939
Exp3 – Gradient Boosting	7.2038	8.4365	0.9940
Selected (Staging) = Exp3	7.2038	8.4365	0.9940

As an independent sanity check, re-scoring the *entire* reference set (73,100 rows) with the registered model gives MAE = 7.01, RMSE = 8.20, and R² = 0.9943, in close agreement with the test-set numbers.

Two caveats about Table 1 should be stated upfront. The performance spread across the three learned models is very narrow (under 0.25 MAE units, under 0.004 in R²), which is smaller than the noise that a random 80/20 split introduces on time-series data (as discussed in Sections 6 and 7). The voting scheme selects Gradient Boosting, but the ranking should be understood as directional rather than definitive: on a different random seed or a time-based split the ordering could easily differ. The meaningful comparator for all three models is the operational baseline in the first row, discussed fully in Section 5.2 below.

5.2 Improving on the operational forecast: a feature-signal analysis

A high R² should always invite the question “*why?*”, and answering it is central to this project’s objective of *beating the operational forecast* rather than merely reproducing it. Table 2 reports the Pearson correlation between each feature and the target on the reference set.

Table 2: Correlation of each feature with the target (*Units Sold*) on the reference set. The operational **Demand Forecast** and **Inventory Level** carry essentially all of the linear signal.

Feature	corr with target
Demand Forecast	0.997
Inventory Level	0.590
Product ID	0.007
Store ID	0.005
Weather Condition	0.005
Discount	0.003
Competitor Pricing	0.001
Price	0.001
<i>(all others)</i>	≈ 0

The picture is clear: the operational **Demand Forecast** is the dominant predictor ($r = 0.997$), **Inventory Level** is moderately informative ($r = 0.59$), and the remaining contextual features carry little marginal linear signal. This is exactly the situation the project set out to probe: the dataset already provides a high-quality forecast column, so the question is not “can we predict demand?” but “can a learned model add anything on top of that existing column?” On clean data the trained model’s output tracks **Demand Forecast** closely (correlation 0.9998), which is the expected behaviour when that column is already near-optimal for the given period.

The honest way to measure the model’s value is therefore against the operational forecast itself, as a baseline. Using **Demand Forecast** directly as the prediction (no learning at all) yields MAE = 8.31 and RMSE = 10.01. The trained pipeline beats this, reaching MAE = 7.01 and RMSE = 8.20 on the full reference set: a roughly one-unit (about 16%) reduction in mean absolute error, achieved by blending the forecast with the contextual features. That improvement is genuine but modest; Section 5.3 then shows that the model nevertheless relies on those contextual features far more than this clean-data number suggests.

5.3 Monitoring behaviour: a clear degradation staircase

Two methodological notes are needed before Table 3 is read. **(i) MAE here measures input sensitivity**, not concept drift: the injection perturbs input features while targets stay at their original values, so the rising error shows how much predictions diverge from ground truth when inputs are corrupted off-distribution, a valid robustness test, discussed fully in Section 6. **(ii) share_drifted_features is a rough directional indicator**: the KS test is applied to all 15 features including integer-encoded categoricals, for which a continuous two-sample test is inappropriate; false positives on those columns explain why some batches flag six or seven features when only four are ever deliberately perturbed.

The monitoring pipeline was exercised across eleven simulated batches of varying size and drift. The resulting **metrics** rows (read directly from the Postgres table via Adminer, Figure 6) are reproduced in Table 3, sorted by drift share so that the trend is visible.

Table 3: All eleven monitoring rows from the **metrics** table, sorted by **share_drifted_features**. As the breadth and severity of injected drift grow, error rises and R^2 collapses: a clear degradation staircase. The two highlighted exceptions are discussed in the text.

batch_id	size	# drifted	share	MAE	R^2
6b0c9215	500	1	0.067	7.02	0.9937
ae2b309a	300	1	0.067	7.33	0.9940
2176c003	200	2	0.133	14.61	0.9614
3bec7ed8	200	3	0.200	<i>6.70</i>	<i>0.9944</i>
94c3271c	200	4	0.267	36.15	0.7803
741b1c85	400	4	0.267	34.46	0.8277
e3f3e39a	400	5	0.333	30.68	0.8439
98d2bad7	500	5	0.333	34.58	0.8170
5952945d	300	5	0.333	31.98	0.8218
3be089b9	300	6	0.400	66.60	0.3792
786ceb3f	500	7	0.467	31.13	0.8434

Language: English PostgreSQL » db » monitoring » public » Select: metrics retail Logout

Adminer 5.4.2

DB: monitoring Schema: public

SQL command Import Export Create table

Select metrics select prediction_logs

Select data Show structure Alter table New item

Select Search Sort Limit 50 Text length 100 Action Select

SELECT * FROM "metrics" LIMIT 50 (0.000 s) Edit

	timestamp	batch_id	batch_size	num_drifted_features	share_drifted_features	mae	rmse	r2	pred_mean	pred_min	pred_max
☐ Modify	2026-05-13 22:27:31.540556	3bec7ed8	200	3	0.2	6.7	7.9044291381478	0.994356808995868	122.93		2
☐ edit	2026-05-13 22:27:34.022217	6b0c9215	500	1	0.0666666666666667	7.018	8.24342161969167	0.993688235998087	138.358		2
☐ edit	2026-05-13 22:27:36.498523	ae2b309a	300	1	0.0666666666666667	7.33	8.40376899571456	0.993971968935727	131.97		2
☐ edit	2026-05-13 22:31:34.987363	2176c003	200	2	0.1333333333333333	14.605	19.8563591828915	0.961389229667245	124.34		3
☐ edit	2026-05-13 22:31:37.494913	786ceb3f	500	7	0.4666666666666667	31.128	41.8338618824512	0.843372268834054	135.132		0
☐ edit	2026-05-13 22:31:40.021106	3be089b9	300	6	0.4	66.59666666666667	84.9119347716601	0.379195177394535	148.44		0
☐ edit	2026-05-13 22:31:42.551402	e3f3e39a	400	5	0.3333333333333333	30.6825	42.4803189724371	0.843931334128557	131.9425		0
☐ edit	2026-06-05 13:18:17.816499	94c3271c	200	4	0.2666666666666667	36.15	49.7729846402644	0.78025169574356	141.6		2
☐ edit	2026-06-05 13:18:25.555083	98d2bad7	500	5	0.3333333333333333	34.58	46.5189423783473	0.817038764130422	134.752		1
☐ edit	2026-06-05 13:18:28.066579	5952945d	300	5	0.3333333333333333	31.98	42.8488817434792	0.821800514968973	121.27		1
☐ edit	2026-06-05 13:18:30.660765	741b1c85	400	4	0.2666666666666667	34.4575	46.3268010982843	0.827721346272844	142.995		2

Whole result 11 rows Modify Selected (0) Export (11) Save

Import

Figure 6: Raw `metrics` table in Adminer (one row per monitoring batch). This is the source of Table 3; alongside drift counts and MAE/RMSE/ R^2 it also stores prediction-distribution statistics (`pred_mean`, `pred_min`, `pred_max`) per batch.

(1) **Drift is detected as designed.** When `generate_batch.py` injects shifts into Price, Discount, Inventory Level, or Competitor Pricing, the KS test flags those (and, through sampling noise, occasionally other) columns at $p < 0.05$, and `share_drifted_features` rises accordingly. Batches with little injected drift report one or two drifted features; the most heavily perturbed batches report six or seven of fifteen.

(2) **Performance degrades sharply with drift.** This is the central empirical result of the monitoring layer, and it is visible at a glance in Table 3 and in the Grafana dashboard (Figure 7). At low drift (share ≈ 0.07) the model retains near-baseline accuracy (MAE ≈ 7 , $R^2 \approx 0.99$). As drift broadens, MAE climbs into the 30s and R^2 falls to ≈ 0.78 – 0.84 ; the worst batch (3be089b9, six drifted features) reaches MAE 66.6 and R^2 0.38, almost a ten-fold increase in error relative to baseline. The monitoring loop therefore does exactly what it is meant to do: it turns an invisible change in the inputs into a visible, quantified loss of predictive quality.

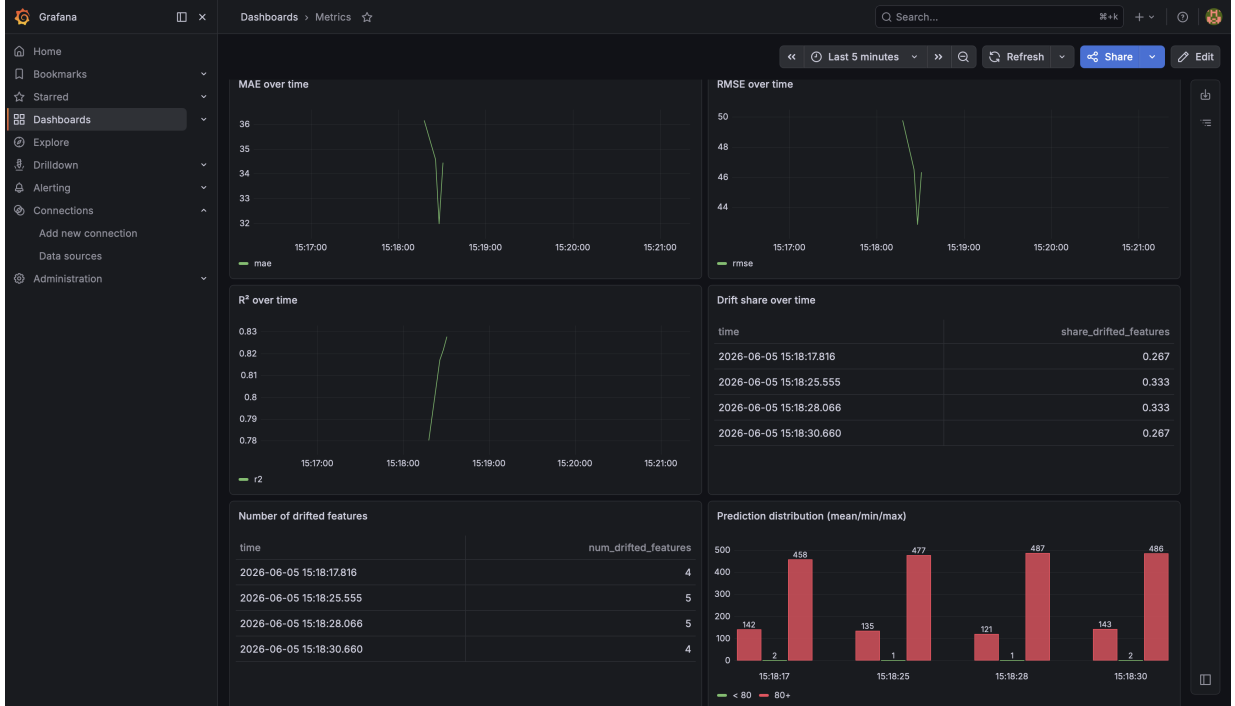


Figure 7: Grafana monitoring dashboard (recent batches). MAE rises into the mid-30s, RMSE into the high-40s, and R^2 falls to ≈ 0.78 – 0.83 as `share_drifted_features` sits around 0.27 – 0.33 and four-to-five features are flagged, giving an at-a-glance view of the degradation seen in Table 3.

(3) **The relationship is strong but not perfectly monotone.** Two batches in Table 3 buck the trend, and they are instructive rather than embarrassing. Batch `3bec7ed8` has three drifted features (share 0.20) yet an essentially baseline MAE of 6.70 ; batch `786ceb3f` has the *most* drifted features (seven, share 0.467) yet a lower MAE (31.1) than the six-feature batch (66.6). The reason is that `share_drifted_features` counts features equally regardless of how much the model relies on them or how severely they were perturbed. A batch can trip the KS test on three low-importance columns while leaving the model-critical `Inventory Level` almost untouched (low error), or trip it on fewer columns but shift `Inventory Level` drastically (high error). The KS share is thus a useful but *unweighted* drift signal, discussed further in Section 6.

5.4 Live prediction logging

Each call to `POST /predict` appends a row to `prediction_logs`, captured in the Adminer view of Figure 8. A representative sample is shown in Table 4. The logged predictions span the full plausible range (from single digits for high-priced furniture to several hundred units for low-priced groceries), and the very first logged row (`S001`, `P0005`, Electronics, North) matches the live API call in Figure 5 exactly (prediction = 146). This closes the loop between the served model and the monitoring layer and provides an audit trail of real traffic.

Language: English PostgreSQL » db » monitoring » public » Select: prediction_logs retail Logout

Adminer 5.4.2

DB: monitoring Schema: public

SQL command Import Export Create table

select metrics select prediction_logs

Select: prediction_logs

Select data Show structure Alter table New item

Select Search Sort Limit 50 Text length 100 Action Select

SELECT * FROM "prediction_logs" LIMIT 50 (0.000 s) Edit

☐ Modify	id	timestamp	store_id	product_id	category	region	price	discount	weather_condition	seasonality	prediction
☐ edit	1	2026-06-05 13:31:30.768143	S001	P0005	Electronics	North	49.99	10	Sunny	Winter	146
☐ edit	3	2026-06-05 13:36:31.054877	S002	P0012	Groceries	South	12.99	5	Rainy	Winter	316
☐ edit	4	2026-06-05 13:36:40.222909	S005	P0031	Clothing	West	59.99	20	Sunny	Spring	74
☐ edit	5	2026-06-05 13:36:45.854338	S003	P0007	Electronics	East	299.99	0	Cloudy	Summer	39
☐ edit	6	2026-06-05 13:36:51.231669	S007	P0019	Toys	North	24.99	15	Sunny	Summer	204
☐ edit	7	2026-06-05 13:36:56.111818	S001	P0044	Furniture	West	499.99	10	Cloudy	Autumn	8
☐ edit	8	2026-06-05 13:37:02.506123	S004	P0003	Electronics	South	149.99	30	Snowy	Winter	85
☐ edit	9	2026-06-05 13:37:07.89144	S006	P0025	Groceries	North	5.49	0	Rainy	Spring	482
☐ edit	10	2026-06-05 13:37:12.911865	S009	P0038	Clothing	East	39.99	25	Sunny	Summer	123
☐ edit	11	2026-06-05 13:37:18.236003	S002	P0051	Toys	West	19.99	10	Cloudy	Autumn	375
☐ edit	12	2026-06-05 13:37:27.551075	S010	P0062	Furniture	South	899.99	15	Snowy	Winter	7

Whole result 11 rows Modify Selected (0) Export (11)

Import

Figure 8: The `prediction_logs` table in Adminer, populated by live FastAPI `/predict` traffic. Each row records the request context and the returned prediction.

Table 4: Representative rows from `prediction_logs` (live API traffic). The logged schema records request context and the prediction, but not the two features that actually drive the prediction.

store	category	region	price	discount	prediction
S001	Electronics	North	49.99	10	146
S002	Groceries	South	12.99	5	316
S005	Clothing	West	59.99	20	74
S003	Electronics	East	299.99	0	39
S006	Groceries	North	5.49	0	482
S001	Furniture	West	499.99	10	8
S010	Furniture	South	899.99	15	7

6 Discussion

What worked. The engineering objective was met in full. Experiments are tracked and comparable in MLflow (Figure 2); the best model is promoted by alias rather than by hard-coded path (Figure 3), so the API, the batch predictor, and the monitoring scripts all consume a single source of truth: `retail_demand_model@Staging` (version 18). The CI/CD pipeline, the project’s headline engineering deliverable, closes the loop on quality: by factoring the prediction logic into a pure, MLflow-free core and testing it with a `DummyModel` against domain-meaningful output constraints, the GitHub Actions suite can run on every push without external dependencies, catching regressions before they reach the served model. Serving, scheduling, containerisation, monitoring, storage, and visualisation are wired together and demonstrably functional, end to end, with live traffic flowing into the same database that the monitoring batches write to. As a demonstration of the MLOps lifecycle on tabular retail data, the project is a success, and its monitoring layer produces a signal that actually *moves* when the world changes.

Read the headline metric against the right baseline. An R^2 of 0.994 looks like a triumph, but on this dataset it mostly reflects an easy distribution: the **Demand Forecast** column is already near-optimal, so any reasonable model inherits a high score. The transferable discipline is to always pair a headline metric with a baseline that represents “what you already had”; doing so here (Section 5.2) turns a potentially misleading 0.994 into an honest, defensible statement of marginal contribution.

Low clean-data contribution does not imply robustness to drift. Sections 5.2 and 5.3 together illustrate a point that is easy to get wrong: a feature adding little marginal accuracy on the clean distribution does not mean the model is insensitive to drift in it. Because **Inventory Level** is collinear with **Demand Forecast** in the training data, the model can lean on it without changing its clean predictions, but that reliance becomes a liability the moment drift decouples the two. This is precisely the kind of failure mode that offline metrics cannot see and that a monitoring loop exists to catch. It also explains why drift detection and performance monitoring are complementary: the per-feature KS test sees the inputs move; the MAE/ R^2 panels see the consequences. In this system the two co-move, but in general they need not: a model could be robust to large input drift (loud KS signal, flat error) or could degrade from concept drift with no input-distribution change at all (silent KS signal, rising error). A monitoring stack needs both signals because either can fire without the other.

Drift count is not drift impact. The non-monotonicity in Table 3 (a three-feature batch with baseline error; a seven-feature batch milder than a six-feature one) exposes a real limitation of **share_drifted_features** as a single health indicator. Counting features with $p < 0.05$ treats a drastic shift in a model-critical feature the same as a trivial shift in an ignored one. A more informative monitor would weight each feature’s drift by its importance to the model; for example, scaling per-feature drift magnitude (a population-stability index, or a Wasserstein/KS *statistic* rather than a binary p -value flag) by the model’s permutation or SHAP importance for that feature. That single change would make the drift score track the error curve far more tightly than the raw count does.

The KS test is the wrong test for several of the “features”. The KS test assumes continuous distributions, yet several monitored columns are integer-encoded *categoricals* (Region, Category, Weather Condition, Seasonality, and the encoded Store/Product IDs). Applying a continuous two-sample test to label codes is statistically improper and can both miss real categorical shifts and raise spurious flags from sampling noise, which partly explains how some batches register six or seven “drifted” features when only four are ever deliberately perturbed. A categorical-appropriate test (chi-squared on the category frequencies, or a population-stability index) should be used for those columns, with the KS test reserved for the genuinely continuous ones (Price, Discount, Inventory Level, Competitor Pricing, Demand Forecast).

The batches simulate covariate drift, not concept drift. The incoming batches are drawn from the later-period data and perturbed on the input side only; the targets are the original, un-perturbed values. This faithfully simulates *covariate* drift (the inputs move) but never *concept* drift (a change in the input→output relationship), which is arguably the more dangerous production failure. Because the targets never move with the inputs, the “ground truth” each batch is scored against corresponds to the *undrifted* world, so the rising MAE measures how far the model’s drifted-input predictions stray from the clean targets, rather than from any new reality. This is the right thing to measure for an input-robustness study, but it should be named as such. It is also worth recording *why* the explicit injection layer was needed at all: a purely temporal split (train on 2022, evaluate on 2023–2024) barely moved any metric, because the synthetic dataset’s feature distributions are near-uniform across years, so without injection there would have been almost no drift to detect, and the monitoring layer’s value could not have been demonstrated.

An observability gap in the prediction log. The **prediction_logs** schema records Store, Product, Category, Region, Price, Discount, Weather, Seasonality, and the prediction, but not

Demand Forecast or **Inventory Level**, the two features that actually drive the output (Table 2). As a result, the live-traffic dashboard can show that predictions vary widely (Table 4) but cannot attribute that variation to its true cause, and a future analyst could not reconstruct a logged prediction from the logged columns alone. Logging the full feature vector (or at least the two dominant features) would make the audit trail genuinely reproducible.

Implication for model choice. Because the operational forecast dominates clean accuracy, the three models perform almost identically on the test set (Table 1: the MAE spread across all three is under 0.25 units), and the voting-based selection is really deciding between near-ties. The win for Gradient Boosting is genuine but marginal, and on a different random seed Random Forest could plausibly take one of the three votes. A more discriminating experiment would either exclude **Demand Forecast** or treat predicting it as a separate upstream task, so that the comparison rewards the model that best uses the *contextual* features rather than the one that best mirrors the operational forecast.

Threats to validity. Three caveats bound the conclusions. (i) The dataset is synthetic, with a **Demand Forecast** column engineered to be near-identical to the target; real demand data would not contain such a feature, so the absolute scores here should not be read as evidence about real-world forecastability. (ii) The *training* split in `pipeline.py` is a random 80/20 split rather than a temporal one (the temporal split is used on the monitoring side), which can leak near-duplicate days across train and test and further inflate the offline score (see Section 7). (iii) The monitoring batches are small (200–500 rows), so the KS test’s power and false-positive rate vary batch-to-batch, adding noise to `share_drifted_features` independent of any real drift.

7 Limitations and Future Work

- **Re-frame the target.** Re-run the comparison *without* **Demand Forecast** to measure how much signal the remaining features actually contain; this is the single change that would most improve the scientific value of the results, and it would also make the drift experiment more realistic (drift would then act on features the model genuinely depends on for its baseline accuracy).
- **Importance-weighted, type-aware drift.** Replace the unweighted KS feature count with a per-feature drift *magnitude* (PSI or KS statistic) weighted by model importance, and use a categorical test for the encoded categorical columns. This directly addresses the non-monotonicity and the improper-test issues raised above.
- **Automated response.** Connect monitoring to action: for example, trigger the Prefect re-training flow or raise an alert when drift or error crosses a threshold, rather than only visualising. The degradation staircase in Table 3 gives a natural place to set such a threshold (e.g. alert when batch MAE exceeds $2\times$ the baseline).
- **Log the full feature vector.** Extend `prediction_logs` to record **Demand Forecast** and **Inventory Level** so that live predictions are fully reproducible and attributable.
- **Harden the existing CI.** Extend the current GitHub Actions suite with an explicit train/serve encoding-consistency check (asserting that the API’s hard-coded category maps match the `LabelEncoder` ordering used at training), in the spirit of the ML production-readiness rubric [1].

8 Conclusion

This project delivers a working, end-to-end machine-learning system for retail demand forecasting, spanning experiment tracking, a model registry, a served API, scheduled retraining, a tested

CI/CD pipeline, and continuous drift and performance monitoring. Framed as the task of *improving on the operational forecast already present in the data*, the deployed Gradient Boosting model succeeds: it beats the “trust the forecast” baseline by roughly 16% in MAE and reaches $R^2 \approx 0.994$ on held-out data. The report’s substantive contribution, however, is the interpretation behind that number. The high score reflects an easy, synthetic distribution rather than the difficulty of the task, so it is read honestly against the operational baseline; and the monitoring experiments show that the learned model is no mere pass-through: it relies genuinely on the contextual features, a reliance that surfaces only when drift is injected and that a single offline metric could never reveal. Read this way, the results validate the *operational* design: a CI/CD-tested service plus a monitoring loop that converts invisible input drift into a clear, quantified degradation staircase, while honestly bounding the modelling claims. They also point to a concrete path forward: re-framing the target away from the operational forecast, importance-weighted and type-aware drift metrics, temporal training validation, and automated retraining triggers, for making the next iteration both more genuinely predictive and even more production-ready.

Tools and Use of AI

Tools. Python 3.13; scikit-learn (`LinearRegression`, `RandomForestRegressor`, `GradientBoostingRegressor`, `StandardScaler`, `LabelEncoder`, metrics); pandas, NumPy, SciPy (`ks_2samp`); MLflow 3.x (experiment tracking, model registry); FastAPI with Uvicorn (prediction service); Pydantic (request validation); Prefect (orchestration and scheduling); Docker and Docker Compose (containerisation); GitHub Actions (CI/CD); pytest with a `DummyModel` stub (unit testing); PostgreSQL (monitoring store); Grafana (dashboards); Adminer (ad-hoc SQL); psycopg (Python–PostgreSQL driver). **Use of AI.** The AI assistant Claude (Anthropic) was used to help structure and debug the pipeline code, and to draft, extend, and refine this report; all methods, results, and interpretations were reviewed and are understood by the author.

References

- [1] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ml test score: A rubric for ml production readiness and technical debt reduction. In *IEEE International Conference on Big Data*, 2017.
- [2] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [3] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.
- [4] João Gama, Indrè Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014.
- [5] Janis Klaise, Arnaud Van Looveren, Clive Cox, Giovanni Vacanti, and Alexandru Coca. Monitoring and explainability of models in production. *ICML Workshop on Challenges in Deploying and Monitoring Machine Learning Systems (arXiv:2007.06299)*, 2020.
- [6] Frank J. Massey Jr. The kolmogorov-smirnov test for goodness of fit. *Journal of the American Statistical Association*, 46(253):68–78, 1951.
- [7] D. Sculley, Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-François Crespo, and Dan Dennison. Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [8] Matei Zaharia, Andrew Chen, Aaron Davidson, Ali Ghodsi, Sue Ann Hong, Andy Konwinski, Siddharth Murching, Tomas Nykodym, Paul Ogilvie, Mani Parkhe, Fen Xie, and Corey Zumar. Accelerating the machine learning lifecycle with mlflow. *IEEE Data Engineering Bulletin*, 41(4):39–45, 2018.